EXTENDED ABSTRACT
ACTIVITY TRACKING WITH THE REACHY MINI ROBOT

Artificial Intelligence
Lectoraat Data Intelligence
Zuyd Hogeschool

Caio de Alcantara Santos (2604500)
Cas Limpens (2400410)
Donné Deguelle (2301667)
Gustavo Gonçalves da Costa (2604824)
Kanak Manjari (Supervisor)
Maarten Vaessen (Client)

2026/2027, Q1
9-11-2026

Abstract

Home-based exercise monitoring could reduce repeated hospital visits, but it requires reliable activity recognition in unconstrained environments. This project investigates whether the Reachy Mini robot can provide person detection and activity monitoring while giving feedback as an exercise buddy. A feasibility prototype was developed using a camera-to-pose-to-classifier pipeline. The system tracks a person, extracts COCO-17 keypoints, computes temporal features, and classifies activities with random forest inference. The shared pipeline can run both in simulation and on the physical robot. MoveNet int8 was selected for the robot because of its hardware constraints, and recognized actions trigger antenna responses. Evaluation used subject-separated public skeleton datasets, external datasets, recorded robot footage, and both full-body and waist-up framing. In real-time evaluation, wave, clapping, and jumping jacks were sufficiently reliable for demonstration. Overall, the results support the Reachy Mini as a feasible CPU-based platform for activity-monitoring research, while showing that this prototype is not a CMAS scorer or a clinically validated decision-support system.

1 Introduction

1.1 Background

The Research Centre for Data Intelligence conducts applied research into, among other themes, smart assistants and human-computer interaction. This project was carried out in the context of the centre's "Measuring at Home with AI" research direction. The long-term motivation for that research is to make exercise monitoring more accessible to children with juvenile dermatomyositis (JDM). These children currently perform exercises related to the Childhood Myositis Assessment Scale (CMAS) under the supervision of specialists at the Wilhelmina Children's Hospital (WKZ). Repeated travel to the hospital is burdensome for children and their families, which creates an opportunity for technologies that can support monitoring at home. Automated home monitoring must do more than record a video. It must detect a person, estimate body movement from visual input, distinguish between activities, and communicate its interpretation clearly to the user. It must also operate under practical conditions, such as changing distances, incomplete views of the body, and limited computing resources. The stakeholders for this feasibility study are the Research Centre for Data Intelligence, the project client and researcher, and the children and families who represent the future users of a home-monitoring system. The project requirements were consolidated in a software requirements specification covering perception, robot interaction, activity recognition, feedback, session management, privacy, accuracy, reliability, usability, and safety. The current project addresses the technical feasibility of such a system; it does not develop or validate a medical assessment instrument.

1.2 Goal

The goal of this project is to investigate whether the Reachy Mini can perform person detection and activity monitoring, including expressive feedback, under the computational constraints of the wireless robot. The resulting artefact is a working prototype that runs in simulation and on the physical robot. It contains a shared camera-to-pose-to-classifier pipeline, temporal feature extraction, activity-recognition models, and robot applications that translate recognized actions into antenna responses. Candidate vision models were benchmarked under the robot's CPU constraints, leading to the selection of MoveNet int8 for on-robot pose estimation. Model-training and export tools were also developed so that the robot can perform inference with NumPy-only model files rather than requiring the full training environment. Recognition models were evaluated on public skeleton datasets with subject or footage-group separation, as well as on robot recordings and an external dataset. Users can also record personalized movements and create their own datasets for future recognition models. The prototype recognizes six classes: none, wave, pushup, squat, clapping, and jumping jacks. Evaluation focuses on technical feasibility using simulation, public skeleton datasets, recorded robot footage, an external test dataset, and different camera framings. The robot's camera should ideally capture the whole person, because waist-up framing removes information that is important for some activities. The prototype is not a CMAS scorer, is not clinically validated, and should not be used as a clinical decision-support system. The central research question is:

> To what extent can the Reachy Mini detect a person, recognize physiotherapy-related exercises, and provide expressive feedback while operating within its computational constraints?

The remainder of this report first discusses the theoretical foundations and related solution space. It then describes the prototype and its evaluation method, presents the results, discusses their implications and limitations, and concludes with directions for future development.

2 Methods

This study employed a custom, iterative methodology to investigate and develop an activity-monitoring system for the Reachy Mini. The approach shares similarities with the Design Science Research (DSR) framework, which is suitable for design-oriented research involving the development and evaluation of an artefact [1]. The approach was selected because the technical feasibility of the system was initially uncertain and several technical choices required investigation before implementation.

The methodology consisted of five phases:

1. Research: identifying stakeholder needs, establishing requirements, and investigating existing techniques.
2. Design: selecting suitable models and defining the technical architecture.
3. Realisation: implementing the activity-recognition pipeline and integrating it with the robot.
4. Testing: evaluating model performance and system behaviour under different conditions.
5. Evaluation: assessing feasibility, identifying limitations, and formulating recommendations.

The phases were not strictly sequential. Research informed design decisions, while implementation and testing were conducted iteratively. Results from experiments were used to refine technical choices and improve the artefact. This approach supported early validation and incremental improvement, which suited the exploratory nature of the project [2]. A traditional waterfall approach was considered less suitable because the technical choices were not fully determined at the start and the project required opportunities to incorporate findings during implementation [3].

Figure 1 illustrates the iterative prototyping approach. Figure 2 shows the activities conducted during the research phase.

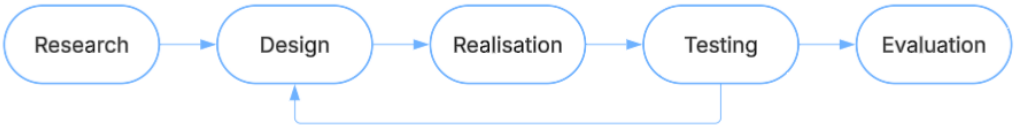


Figure 1: Iterative prototyping approach used in the project.

2.1 Requirements and software requirements specification

The research began with a stakeholder analysis to identify the parties involved and their interests and needs. Functional and non-functional requirements were derived from the project description and meetings with the client. These requirements were consolidated in a software requirements specification (SRS), which served as the reference for design and evaluation.

The functional requirements cover perception and local processing, robot behaviour and interaction, activity and exercise recognition, assessment and feedback, and session management. The non-functional requirements address privacy, recognition accuracy, reliability, usability, and safety. The complete SRS contains the detailed requirement statements and is included as an attachment to the article.

The research was organised around two complementary activities. First, existing techniques for person detection, pose estimation, and activity recognition were investigated. Second, relevant scientific literature, datasets, standards, and guidelines were studied to identify suitable approaches and relevant constraints. The findings were compared against the system requirements and the Reachy Mini's camera and computational capabilities. This process informed the selection of candidate models and the subsequent evaluation of their suitability.

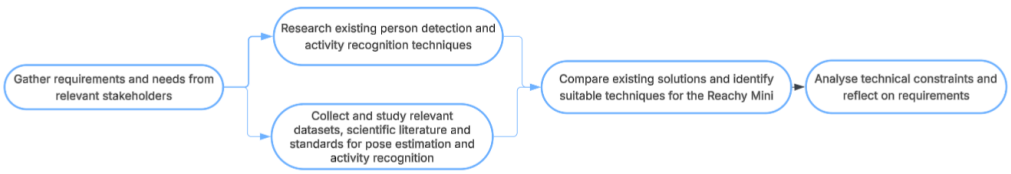


Figure 2: Research activities used to establish requirements and identify suitable techniques for the Reachy Mini.

2.2 Pose-model research and selection

Candidate models for human pose detection and temporal activity analysis were investigated under the Reachy Mini's CPU and camera constraints. The candidates included BlazePose for fast 2D pose detection, ViTPose as a transformer-based 2D estimator, WHAM and CameraHMR for 3D motion or pose-and-shape estimation, MoveNet for lightweight 2D pose estimation, SAM2 for segmentation, PySlowFast for video classification, FSPose for fused-heatmap pose estimation, GraphEnet for graph-based pose estimation, YOLOPose for pose estimation using the YOLO framework, MoViNets for efficient video recognition, and TSM-MobileNetV2 for lightweight temporal modelling.

BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose, YOLOPose, and MoveNet were benchmarked under comparable conditions. Throughput and mean pose-inference time were recorded. The evaluation also considered whether each model could run on the robot and produce the COCO-17 keypoint representation required by the downstream feature-extraction and recognition pipeline.

MoveNet Lightning int8, executed through LiteRT, was selected because it provided the most suitable trade-off between speed, keypoint compatibility, and deployment feasibility.

2.3 Training, export, and inference

No new video recordings were collected for model training. Instead, the recognition models were trained using existing COCO-17 skeleton data published by OpenMMLab for NTU RGB+D 60 [4] (wave and clapping) and UCF101 [5] (pushup, squat, and jumping jacks). HMDB51 was used only as an external test set. All other dataset classes were labelled as none.

Each clip was resampled from 30 to approximately 10 FPS to match the robot camera. The clips were then divided into temporal windows using the same feature-extraction code as the live system: 1.5-second windows with nine features for the binary wave model, and 3-second windows with 34 features for the six-class action model. To approximate the table-mounted camera perspective, each clip was also windowed with leg and hip keypoints removed.

Random forests were trained using scikit-learn [6]. The wave model used 100 trees with a maximum depth of six, while the action model used 300 class-balanced trees. The models were evaluated using five-fold GroupKFold by subject or footage group, following [7]. Precision, recall, and F1 scores were calculated from out-of-fold predictions. A confidence threshold was tuned for each class using these predictions; windows below the threshold were classified as none.

The final forests were fitted on all training windows and exported to compressed `.npz` files containing the decision-tree node arrays. On the robot, a NumPy-only implementation traverses these trees and reproduces scikit-learn's `predict_proba` behaviour, removing the need for scikit-learn during inference.

2.4 Robot integration

The applications use the Reachy Mini Python SDK (`reachy_mini` 1.10.0, matched to the daemon version) for motion and the daemon REST API on port 8000 for state management.

Before camera, tracking, or movement operations, a standard-library preflight procedure checks network reachability, daemon and backend state, running applications and movements, motor mode, sleep pose, camera ownership, and tracker frames. When automatic correction is enabled, the procedure enables the motors, executes the wake-up movement, and reacquires the camera. It returns an explicit ready, not-ready, or unreachable status. A context manager combines preflight checks, SDK connection, and cleanup, including stopping head tracking and optionally returning the robot to its sleep pose.

Because the vision loop operates at approximately 10 FPS while the daemon controls the robot at 50 Hz, a dedicated thread streams `set_target` commands at 50 Hz. Each recognised action produces a distinct antenna movement pattern defined by amplitude, frequency, and a whole number of cycles ending at the neutral pose. Head and body following use an integrating controller based on the image-position error of the torso centre. Body yaw extends the tracking range beyond the head's 60-degree movement limit relative to the body.

Additional integrations include an annotated MJPEG camera stream for browser-based observation without a local display, a remote-inference mode that sends low-resolution frames (192×108 pixels) to a laptop GPU and receives keypoints over HTTP, and a coach workflow for training personalised models on the robot's CPU. In the coach workflow, scikit-learn runs in a low-priority subprocess to keep the camera and tracking loops responsive. A failed training run does not overwrite the previous model. Deployment uses an isolated virtual environment to avoid modifying the robot's system environment.

2.5 Testing and evaluation

Testing and evaluation examined the technical feasibility of the system against the requirements established during the research phase. Model performance was assessed using the benchmark results and the out-of-fold precision, recall, and F1 scores. HMDB51 provided an external test set beyond the datasets used for training. Camera-framing experiments considered the effect of removing hip and leg keypoints on the information available for activity recognition.

The evaluation also considered inference speed, CPU-only deployment, compatibility with the Reachy Mini's camera and movement interfaces, and system responsiveness during simultaneous recognition and tracking. The findings were used to identify technical limitations and formulate recommendations for further development. The project focused on technical feasibility; it did not validate a clinical assessment instrument or establish clinical suitability for physiotherapy.

3 Results

3.1 Description of the Artefact

The end product is a software application: a working prototype (an instantiation, in Design Science Research terms) that turns the Reachy Mini into a CPU-only activity monitor that answers what it sees with its own body. It consists of three parts: a layered software architecture that runs the same recognition code in simulation and on the robot, recognition models exported as NumPy-only files, and robot applications built on both. It addresses the requirements of the software requirements specification (SRS) attached to this article (Section 3.1).

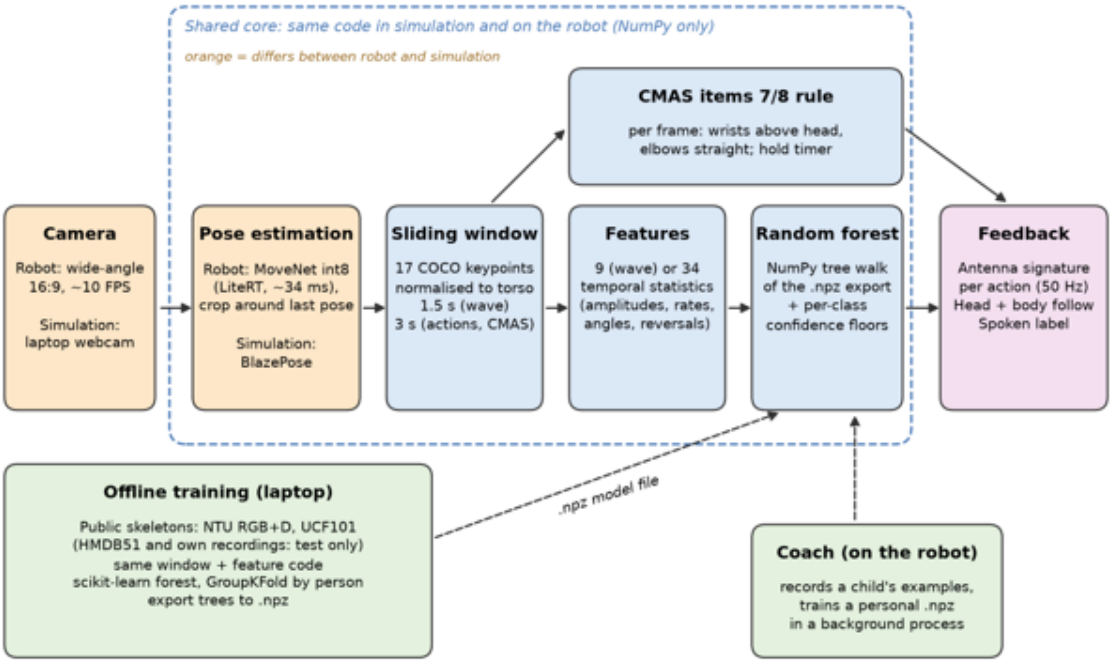


Figure 3: Pipeline Architecture.

Figure 3 represents the artefact's pipeline. Everything inside the dashed line is the same code in simulation and on the robot; only the camera and the pose estimator differ. Models are trained offline, or by the coach on the robot, and loaded as `.npz` files.

The pipeline works in such way that a camera frame goes to a pose estimator that returns 17 COCO keypoints: MoveNet Lightning int8 on the robot, BlazePose on the laptop webcam in simulation. The keypoints enter a sliding window, are normalised to the torso and summarised as temporal features. A random forest, evaluated by a NumPy-only tree walk, classifies every window, and a per-class confidence floor turns uncertain windows into *none*. Each detection drives the feedback: a distinct antenna signature per action, rendered at 50 Hz independently of the 10 FPS vision loop, head and body following that keeps the person in view, and an optional spoken label.

As for architecture and organization, the code is organised by where it must run: a shared core (pose backends, features, models, feedback), robot applications, simulation tools for MuJoCo and a webcam, and training tools, which are the only part that needs scikit-learn. Dependencies point only into the core, so one model file and one feature implementation serve both environments, and training and inference cannot drift apart.

For the final application, the robot runs action recognition (the six classes, or the CMAS items of Section 4.6), wave recognition, head and body tracking, and a coach for children. The coach guides a child through about 4.5 minutes of recording (squats, arm raises, side bends), trains a personal model on the robot's own CPU while the head keeps following, and then runs a workout with that model. A preflight tool brings the robot into a known state before every application.

Table 1 links the main decisions to the requirement or insight behind them.

3.2 Pose-Model Performance

The benchmark compared BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose and MoveNet as candidates for the perception pipeline. MoveNet Lightning int8 reached

Table 1: Origin of the main design decisions.

Origin	Decision
Requirement: local processing on a CPU without GPU	MoveNet int8 for pose; forests exported to NumPy instead of a deep skeleton network
Requirement: privacy of child users	Only keypoints are stored; children's recordings and models stay on the robot
Grounding: windowed data leaks between splits	Every score from GroupKFold by person
Grounding: public datasets may not transfer to a room	Robot and room recordings kept as external test sets only
Design cycle: BlazePose cannot run on the robot's CPU (its binary needs AES instructions)	Pose backend made swappable behind one interface
Design cycle: a table-mounted camera often sees the person waist-up	Training windows also built with hips and legs removed
Design cycle: letterboxing the 16:9 wide-angle frame left the person a few dozen pixels tall	Pose crop centred on the previous pose; person detected in 72% of frames instead of 52%
Design cycle: scikit-learn inference cost ∼33 ms per window on the robot	Trees exported to `.npz` and walked in NumPy
Design cycle: the robot often boots asleep, motors off, camera released	Preflight with automatic fix

23.80 FPS in the inference benchmark, with a mean pose-inference time of approximately 34 ms on the Reachy Mini; in the complete application the camera, at about 10 FPS, sets the rate (Section 4.5). This was the most suitable result for the CPU-only target, and the model produces the COCO-17 keypoints the recognition code expects. Cropping each frame around the previous pose, instead of letterboxing the whole wide-angle frame, raised the share of robot frames with a detected person from 52% to 72%. On a replay of a laptop recording with the robot's aspect ratio, it raised the mean keypoint confidence from 0.48 to 0.68.

3.3 CMAS Items

In CMAS mode the prototype recognises that four CMAS items were performed; it does not score them. Items 12 (chair rise) and 14 (pick-up) are classified by a three-class forest (none, chair rise, pick-up) trained on NTU RGB+D alone (actions "standing up" and "pick up"). Items 7 and 8 (arms straight above the head, and for how long) use a geometric rule on single frames, because a held posture has no movement for a window to learn. Item 13 (stool step) was left out: no public dataset contains it and none was recorded.

3.4 Pose-model performance

The benchmark compared BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose and MoveNet as candidate components for the perception pipeline. MoveNet Lightning int8 achieved an average of 23.80 FPS, with a mean pose-inference time of approximately 34 ms on the Reachy Mini. This was the most suitable result for the project's CPU-only deployment target. The model also produces the COCO-17 keypoints expected by the temporal feature and recognition code.

The comparison also exposed deployment constraints. BlazePose and ViTPose are useful comparison models, but the robot cannot use the available BlazePose MediaPipe binary because it requires AES instructions unavailable on the Reachy's Cortex-A72; ViTPose's detector-and-pose pipeline is intended for the laptop or simulation environment. MoViNet-A0 and TSM-MobileNetV2 were considered for temporal classification rather than selected as replacements for the keypoint-based pipeline. Consequently, MoveNet int8 was used for robot inference, while the shared downstream representation remained model-independent.

3.5 Wave recognition

On two recorded sessions with our own robot (416 windows), the rule-based detector reached F1 0.811 (precision 1.000, recall 0.682). The NTU-trained forest, which never saw these recordings, reached F1 0.951 and outperformed a forest trained on the sessions themselves (F1 0.884, historical only). On unseen NTU subjects, where the negatives include deliberately

confusable gestures (salute, selfie and brushing hair), F1 was 0.556. On HMDB51 movie clips it was 0.065, a deliberately pessimistic external check.

3.6 Six-class action recognition

Table 2: Out-of-fold F1 on held-out subjects or footage groups. Waist-up framing is simulated by removing the hip, knee and ankle keypoints.

Class	Full body	Waist-up
none	0.902	0.855
wave	0.918	0.873
squat	0.922	0.803
clapping	0.898	0.899
jumping_jacks	0.887	0.861
pushup	0.381	0.137

These are in-domain scores. A source confound inflates wave and clapping, both taken from the studio-recorded NTU corpus. On our robot recordings wave drops to 0.581, the figure to quote for a robot in a room, and clapping has not yet been measured on robot footage. Squats are almost never false positives (precision 0.998) but miss about one repetition in seven (one in three waist-up). Push-ups (19 training groups, per-fold F1 0.06–0.71) are insufficiently measured and are not demonstrated. Without crop augmentation, waist-up F1 for wave, squat and clapping fell to 0.000.

4 Discussion

Several choices contributed positively to the results. Training on skeletons from public datasets gave the project a fast start: working recognition models existed before any data of our own had been recorded, and no video had to be processed. That head start had a limit, however. The public datasets did not cover every exercise the project needed, and they did not show a person the way the robot sees one, so we later had to record our own sessions to train and test the models that mattered most. A second positive choice was to run the same code in simulation and on the robot. Features, models and feedback were developed and checked on a laptop, then deployed to the robot unchanged, which kept the two environments from drifting apart. Finally, short design cycles on the physical robot exposed problems that no laptop test would have shown. They revealed that BlazePose cannot run on the robot's processor, and that letterboxing the wide-angle image left the person too small to detect reliably. Each problem was found and fixed within days, and the crop around the previous pose raised the share of frames with a detected person from 52 percent to 72 percent.

Other aspects limited the results. The robot's camera does not match the image quality of a laptop or phone camera. Its wide-angle view leaves a person only a small part of the image, so keypoints are less certain than in simulation. The CPU, without a GPU, also restricted the choice to a lightweight pose model. A further limit was the difference between the training videos and real use. The public datasets were recorded in studios or taken from films, with other backgrounds, camera angles and sometimes several people in view. The robot, in contrast, sees one person in a home room from table height. This difference lowered recognition quality outside the datasets: waves, for example, scored F1 0.918 on dataset subjects but 0.581 on our robot recordings. The robot was also not always available to the team. The simulation largely mitigated this risk, because most of the development could continue without it. Lastly, engagement could not be measured, because the prototype could not be tested with real users. Whether children find the robot an engaging exercise buddy therefore remains an open question.

5 Conclusion

The goal was to find out whether the Reachy Mini can detect a person, recognise their activity and give expressive feedback within the limits of its own CPU. For the technical part of that goal the answer is The complete pipeline runs on the robot's Raspberry Pi CM4 without a GPU or a laptop. Pose estimation takes about 34 ms per frame, features and forest a few milliseconds, and the camera's 10 FPS, not the computation, sets the pace. The same code and models run in simulation, so they are developed on a laptop and deployed unchanged. Recognised actions are answered in real time with antenna signatures and head and body following.

The answer to "how well" is more limited. In live use, wave, clapping and jumping jacks were reliable enough to demonstrate, and squats were recognised with about one repetition in seven missed. Push-ups were not. Scores on public datasets overstate what happens in a room: the six-class model recognises waves with $F_1 = 0.918$ on dataset subjects but $F_1 = 0.581$ on our robot recordings. For the CMAS items, pick-up held up in a room ($F_1 = 0.933$). Chair rise and arms above the head were recognised live on the robot, but they have not yet been measured on a correctly performed room recording. The robot's position also matters: it must see the whole person, and a waist-up view costs recognition.

The prototype, its models, the training and export tools, and the coach were realised. A CMAS score, the stool-step item and clinical validation were not; they were outside the scope. The "engaging" part of the research question was not evaluated, since no user study was done. The prototype is therefore a research platform: it shows that monitoring at home with this robot is technically feasible, but it cannot yet be used to monitor children with JDM.

That said, there are four recommended next steps:

1. Record labelled sessions in real rooms through the robot's own camera, following the CMAS instructions, and use them as the reference for room performance.
2. Collect data for the classes that lack it: push-ups, clapping on robot footage, and the stool step.
3. Run a small user study with children and families to measure engagement, which the research question assumes, but this project did not test.
4. If CMAS scoring becomes the goal, define the scoring criteria with the WKZ specialists and validate the system against their scores.

References

[1] A. R. Hevner, "A three cycle view of design science research," *Scandinavian Journal of Information Systems*, vol. 19, no. 2, pp. 87–92, 2007. [Online]. Available: https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1017&context=sjis

[2] GeeksforGeeks. "What is prototyping? definition, types, qualities and more," Accessed: Oct. 26, 2026. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/what-is-a-prototype-and-how-to-create-it/

[3] Agile Scrum Group. "Agile vs waterval: Wanneer gebruik je welke methode?" Accessed: Oct. 26, 2026. [Online]. Available: https://agilescrumgroup.nl/agile-vs-waterval/

[4] A. Shahroudy, J. Liu, T.-T. Ng, and G. Wang, "Ntu rgb+d: A large scale dataset for 3d human activity analysis," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 1010–1019. DOI: 10.1109/CVPR.2016.115

[5] K. Soomro, A. R. Zamir, and M. Shah, "Ucf101: A dataset of 101 human actions classes from videos in the wild," Center for Research in Computer Vision, University of Central Florida, Tech. Rep. CRCV-TR-12-01, 2012. arXiv: 1212.0402.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011.

[7] S. Saeb, L. Lonini, A. Jayaraman, D. C. Mohr, and K. P. Kording, "The need to approximate the use-case in clinical machine learning," *GigaScience*, vol. 6, no. 5, pp. 1–9, 2017. DOI: 10.1093/gigascience/gix019

Appendices

Appendix A: System Requirements Specification

M-ADSAI-ZM002-26

System Requirements Specification

Caio De Alcantara Santos (2604500), Cas Limpens (2400410), Donné Deguelle (2301667) and Gustavo Gonçalves Da Costa (2604824).

Contents

Introduction 3
Purpose 3
Scope 3
Intended Users 3
System Overview 4
Product Description 4
Users and Stakeholders 4
Assumptions and Dependencies 4
Functional Requirements 5
Perception and local processing 5
Robot behaviour and interaction 5
Activity and exercise recognition 6
Assessment and feedback 6
Session management 7
Non-Functional Requirements 8
Bibliography 9

Introduction

Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for a system that uses the Reachy Mini robot and AI-based movement detection to monitor and evaluate human movements during physiotherapy exercises at home. The project investigates whether the Reachy Mini can be used as an interactive and motivating exercise buddy. The system should be able to detect and potentially score common human movements and provide feedback or encouragement to the user. The SRS defines the functional and non-functional requirements of the proposed system and serves as a basis for its development and evaluation.

Scope

The scope of this project is to investigate the capabilities of the Reachy Mini for person detection and activity monitoring. The system will focus on detecting common movements such as walking, running, sitting, standing, and other relevant physiotherapy movements. The project will investigate whether existing AI and pose-estimation models can identify the movement being performed, determine whether it is performed correctly, and potentially provide a score. The Reachy Mini may also provide interactive feedback through movements, sounds, or voice to increase motivation. The project does not aim to solve a specific medical case or provide a complete medical monitoring solution. Instead, the focus is on investigating whether the Reachy Mini can identify and score general human movements and function as a motivating device for home-based physiotherapy.

Intended Users

The primary intended users are patients performing physiotherapy exercises at home. The system is intended to support users who perform exercises independently by providing motivation and potentially feedback on their movements. The Reachy Mini is intended to act as an interactive exercise buddy that can react to the user's movements and provide encouragement or feedback. The system may also be relevant to physiotherapists and healthcare professionals, who could use the Reachy Mini as a replacement assistant for appointments and scoring, to support the monitoring of patients' exercises. The original project document specifically mentions children with JDM (Juvenile Dermatomyositis) performing exercises that are assessed using the CMAS (Childhood Myositis Assessment Scale), but this project broadens the current project goal toward general physiotherapy movements and investigating the Reachy Mini as a motivating device.

System Overview

Product Description

The product is a movement detection system that uses the Reachy Mini robot's capabilities and AI algorithms to detect and score JDM patients movement during physiotherapy exercises at home. The system aims to provide motivation to patients. To support exercise performance.

Users and Stakeholders

The intended users of the Reachy Mini are JDM patients. But also, healthcare professionals who assess these patients. The robot could help assess the patients, without the patients needing to travel to the hospital as often.

Our client Maarten Vaessen who has tasked us with this project, is a senior researcher in AI for healthcare at Zuyd research center data intelligence (Zuyd University of Applied Sciences, n.d.).

Assumptions and Dependencies

The Reachy Mini can be programmed with the use of the Reachy Mini python SDK. Which is supported on Windows, MacOS and Linux. The python SDK requires python version 3.10-3.12 and the latest version of git, including git LFS (Huggingface, n.d.). It is assumed that the client will provide the Reachy Mini DIY kit.

Functional Requirements

The functional requirements describe what the system has to do. Each requirement has a unique identifier, is written using the modal verb *shall*, and applies to a single user in front of a single Reachy Mini during an exercise session.

Several requirements refer to *supported activities*. This means the closed set of movements the system is able to recognise: standing, sitting, walking, running, arm raise and sit-to-stand. The set is deliberately small, because the aim of the project is to find out whether recognition of this kind is feasible on the Reachy Mini, not to cover the full range of physiotherapy exercises.

Perception and local processing

Before an activity can be recognized, the system must detect and track the user. The activity recognition approach may either use an explicit skeletal or pose representation or infer activities directly from image or video data. Both approaches are therefore considered within the scope of this research. FR-02 additionally requires that this happens locally, which is the constraint that gives the project its research character and from which the privacy property in NFR-01 follows.

Table 1 – Perception and local processing functional requirements

ID	Title	Requirement
FR-01	Person Detection	The system shall detect a person within the Reachy Mini's camera view within 5 seconds after the person enters the camera view during an exercise session.
FR-02	Local Processing	The system shall process captured camera data entirely on the Reachy Mini or the connected local computer, without requiring external processing for core activity-tracking functionality.
FR-03	Person Tracking	The system shall track the detected person continuously while the person remains visible within the camera's field of view during an exercise session.
FR-05	Pose Estimation	The system shall provide the input representation required by the selected activity recognition approach, which may be an explicit pose/skeletal representation or image/video data.
FR-18	User Detection Loss	The system shall detect that the user is no longer visible to the camera within 2 seconds after the user leaves the camera's field of view during an exercise session.

Robot behaviour and interaction

The robot moves for two different reasons. In FR-04 the head moves in order to keep the user in view, which is a perception function. In FR-13 and FR-14 the head and antennas move in order to communicate with the user, which is an interaction function. How these two influence each other is one of the questions the project sets out to answer.

Table 2 – Robot behaviour and interaction functional requirements

ID	Title	Requirement

FR-04	User Following	The system shall move the Reachy Mini's head to follow the detected user while the user remains within the camera's field of view during an exercise session.
FR-12	Audio Feedback	The system shall provide audio feedback through the Reachy Mini's speaker whenever an audio feedback response is triggered.
FR-13	Head Movement Feedback	The system shall use movements of the Reachy Mini's head to provide feedback to the user whenever a head-movement feedback response is triggered.
FR-14	Antenna Movement Feedback	The system shall use movements of the Reachy Mini's antennas to provide feedback to the user whenever an antenna-movement feedback response is triggered.

Activity and exercise recognition

Three categories are distinguished because they place different demands on the recognition approach. Static positions can be classified from a single pose, dynamic movements require a sequence of poses over time, and physiotherapy exercises additionally have a defined correct execution.

Table 3 – Activity and exercise recognition functional requirements

ID	Title	Requirement
FR-06	Static Movement Recogntion	The system shall recognise all static positions listed as supported activities during an exercise session.
FR-07	Dynamic Movement Recognition	The system shall recognise all dynamic movements listed as supported activities during an exercise session.
FR-08	Physiotherapy Exercise Recognition	The system shall recognise all physiotherapy exercises listed as supported exercises during an exercise session.

Assessment and feedback

Recognising a movement is not enough on its own, since the robot is meant to act as an exercise buddy. FR-09 and FR-10 cover the judgement about the execution, and FR-11 covers the decision to respond. The assessment is explicitly non-clinical: the movement criteria describe geometric properties such as joint angles and symmetry, and do not constitute a medical score.

Table 4 – Assesment and feedback functional requirements

ID	Title	Requirement
FR-09	Movement Assessment	The system shall assess the execution of supported physiotherapy exercises using the predefined movement criteria for each exercise.
FR-10	Repetition Detection	The system shall detect completed repetitions according to the predefined repetition criteria during an exercise session.

Session management

An exercise session is the unit of operation of the system: the user starts it, the system observes continuously during it, and the results are reported at the end.

Table 5 – Session management functional requirements

ID	Title	Requirement
FR-11	Session Management	The system shall allow the user to start and end an exercise session.
FR-12	Exercise Monitoring	The system shall monitor the user's movements continuously throughout an active exercise session.
FR-13	Exercise Result	The system shall provide the user with the detected exercise results and assessment results at the end of an exercise session.

Non-Functional Requirements

The non-functional requirements describe the qualities the system has to exhibit rather than the functions it has to perform. Each is expressed as a measurable target, so that the outcome of the investigation can be reported objectively.

Privacy is a driving constraint rather than a secondary concern, because the system observes a patient exercising at home. Accuracy determines whether the system is useful at all, since feedback based on a misrecognised movement is worse than no feedback; the threshold of 80% demonstrates technical feasibility within the scope of this project and is not a level that would be sufficient for clinical use. Reliability and usability follow from the fact that the system is operated by a patient without technical support present, and safety follows from the robot moving in the physical presence of a user who may be a child.

Table 6 – Non-Functional Requirements

ID	Title	Requirement
NFR-01	Privacy	The system shall ensure that captured video and session data remain on the Reachy Mini and/or the connected local computer for the entire duration of system operation and shall not be transmitted to external systems.
NFR-02	Recognition Accuracy	The system shall provide activity recognition for the supported activities when evaluated using the predefined test dataset. The recognition performance shall be measured and reported.
NFR-03	Reliability	The system shall complete an exercise session without requiring an application restart under normal operating conditions.
NFR-04	Usability	The system shall allow first-time users to start an exercise session without assistance from the development team.
NFR-05	Safety	The system shall not perform unexpected or uncontrolled robot movements during normal operation.

Bibliography

Huggingface. (n.d.). *Installation Guide*. Retrieved from huggingface.co: https://huggingface.co/docs/reachy_mini/SDK/installation?install-uv=Windows&install-git=Windows&install-method=%F0%9F%93%A6+Option+A%3A+PyPI

Zuyd University of Applied Sciences. (n.d.). *Data intelligence*. Retrieved from zuyd.nl: https://www.zuyd.nl/en/research/centres/data-intelligence